SonarQube Integration with CI

- Objective
- Purpose
- System Requirements
- Installing SonarQube
- Installing Language Plugins
- Updating the Plugins
- Analyzing Source Code with SonarQube Scanner
- Customizing the Rule Sets
- Integrating SonarQube with Jenkins CI
- Impact Analysis

Objective

Objective of this page is to document the steps required to install and integrate SonarQube static code analysis tool with Jenkins CI server. This will enable us to automate the static code analysis process.

Summary of all previous code analysis reports: Static Code Analysis Using SonarQube

Purpose

Integration of SonarQube with CI pipeline will help us in multiple ways:

- Automate the static code analysis process, so that it can be triggered based on an event (e.g. after merging a branch to Next)
- Integration with Huell (Mashery Bot courtsey of DevOps team) will enable engineering team to run static code analysis on a desired branch and compare the results with baseline.
- It will help us in making sure that no new tech debt / vulnerabilities are added to the codebase
- It will also help us in improving the code quality on a continuous basis across engineering team for langauages like Java, PHP and JavaScript.

System Requirements

S. No	Software/Plugin	Version	New or Exist in Infrastructure	License	Network Support like Port/Firewall	Notes
1	SonarQube	5.4	New	LGPL v3	Port 9080 (or any other port) on which SonarQube needs to be hosted	
2	MySQL	5.7	Existing		NA	Only InnoDB storage engine is supported, not MyISAM
3	SonarQube PHP Plugin	2.6	New	GNU LGPL v3	NA	
4	SonarQube Java Plugin	3.5	New	GNU LGPL v3	NA	
5	SonarQube JavaScript Plugin	2.8	New	GNU LGPL v3	NA	
6	SonarQube Jenkins Plugin	2.3	New	GNU LGPL v3	NA	

Installing SonarQube

1. Make sure that all the requirements are met before installing SonarQube. http://docs.sonarqube.org/display/SONAR/Requirements
2. Install MySQL server. For more details on installing MySQL, please refer http://dev.mysql.com/doc/refman/5.7/en/installing.html
3. Download Sonar from the download link or download a native package instead
4. Configure the database access:
    a. Locate the configuration file at <install_directory>/conf/sonar.properties. This config file contains all the configurations, we just need to uncomment and modify a few configurations as mentioned below.
    b. Database user credentials:

```
# User credentials.
# Permissions to create tables, indices and triggers must be granted to JDBC user.
# The schema must be created first.
sonar.jdbc.username=<username>
sonar.jdbc.password=<password>
```

    c. Database connection string:

```
#----- MySQL 5.x
# Only InnoDB storage engine is supported (not myISAM).
# Only the bundled driver is supported.
sonar.jdbc.url=jdbc:mysql://localhost:3306/sonar?
useUnicode=true&characterEncoding=utf8&rewriteBatchedStatements=true&useConfigs=maxPerformance
```

5. By default, Sonar runs on port 9000. This can be changed by modifying below lines in config file:

```
# TCP port for incoming HTTP connections. Disabled when value is -1.
sonar.web.port=9080
```

6. If the server has more than one IP addresses, then one IP address can be bound with the server:

```
# Binding IP address. For servers with more than one IP address, this property specifies which
# address will be used for listening on the specified ports.
# By default, ports will be used on all IP addresses associated with the server.
sonar.web.host=XXX.XXX.XX.XX
```

7. Start the server by using below command:

Linux/Mac OS

```
<install_directory>/bin/<YOUR OS>/sonar.sh start
```

Windows

```
 <install_directory>/bin/windows-x86-XX/StartSonar.bat
```

8. SonarQube can be browsed now at *http://ip-address:port* The default administrator credentials are admin/admin:

```
http://XXX.XXX.XX.XX:9080
```

For more details on installing SonarQube, please refer http://docs.sonarqube.org/display/SONAR/Installing+the+Server

Installing Language Plugins

In order to analyze the PHP, JavaScript and Java projects, the language specific plugins need to be installed. To install these plugins, below steps need to be followed:

1. Login to Sonar dashboard with admin account
2. Click on "Settings" option frmo top menu
3. Go to System Update Center
4. This will show the list of installed and available plugins
5. Click on "Available Plugins" and it will list all the plugins that can be installed and used
6. Navigate to the "Languages" section in the list
7. Click on the language name and then click on "Install" button
8. This will install the plugin and ask for restarting the Sonar server if required
9. After restarting the server, "Installed Plugins" list will show the newly installed plugin
10. Below are the links to home pages of plugins that are being used for scanning PHP, JavaScript and Java code repositories:
    a. PHP: http://docs.sonarqube.org/display/PLUG/PHP+Plugin
    b. JavaScript: http://docs.sonarqube.org/display/PLUG/JavaScript+Plugin
    c. Java: http://docs.sonarqube.org/display/PLUG/Java+Plugin

Updating the Plugins

In order to keep the plugins updated, follow the below steps:

1. Follow steps 1 to 4 mentioned in the above section
2. Click on "Plugin Updates", this will show the list of all plugins for which update is available
3. The plugin can be updated by clicking on "Upgrade to X.XX" where X.XX will show the new version which is available

Note

1. When a language plugin is upgraded, it adds more rules to the system. We need to go through the newly added rules and take appropriate action (e.g. including/excluding rule from the quality profile)
2. This will also impact on the baseline due to addition of new rules, so the baseline also need to be updated after upgrading the language plugin

Analyzing Source Code with SonarQube Scanner

There are multiple ways of analyzing the source code, we are using the SonarQube Scanner for the same. Below are the steps to install and configure SonarQube Scanner and analyzing source code using it:

1. Download the latest version from SonarQube Scanner home page
2. Extract the downloaded file into a directory say <install_directory>
3. Configure the settings in <install_directory>/conf/sonar-runner.properties file
    a. Set the SonarQube server URL:

```
 #----- Default SonarQube server
sonar.host.url=http://localhost:9080
```

    b. Set the database connection string:

```
#----- MySQL
sonar.jdbc.url=jdbc:mysql://localhost:3306/sonar?useUnicode=true&amp;characterEncoding=utf8
```

    c. Set the database user credentials:

```
#----- Global database settings
sonar.jdbc.username=sonar
sonar.jdbc.password=sonar
```

4. Verify the installation by running:

```
<install_directory>/bin/sonar-runner -h
```

   It should show the output as below:

```
usage: sonar-runner [options]

Options:
 -D,--define <arg>     Define property
 -e,--errors           Produce execution error messages
 -h,--help             Display help information
 -v,--version          Display version information
 -X,--debug            Produce execution debug output
```

5. Create a configuration file named *sonar-project.properties* in the root directory of the project. Below is a sample configuration file:

sonar-project.properties

```
# Required metadata
sonar.projectKey=<Unique key for this project>
sonar.projectName=<Name of project, This will be visible on dashboard>
sonar.projectVersion=<Version of the project>

# Path to the parent source code directory.
# Path is relative to the sonar-project.properties file. Replace "\" by "/" on Windows.
# Since SonarQube 4.2, this property is optional if sonar.modules is set.
# If not set, SonarQube starts looking for source code from the directory containing
# the sonar-project.properties file.
sonar.sources=<optional | This should include all the directories that need to be included in the scan.>

# Language
sonar.language=<PHP | JavaScript | Java>

# Encoding of the source code
sonar.sourceEncoding=UTF-8

# Additional parameters
# sonar.my.property=value

sonar.exclusions=<optional | This should include the list of files / directories to exclude from scan>
```

6. To run code analysis on a project, navigate to the project directory and run:

```
<sonar_runner_install_directory>/bin/sonar-runner
```

7. Once the Sonar Runner task is complete, then the project will be visible on SonarQube dashboard with the value set for "sonar.projectName"
8. For more details on Analyzing Source Code, please refer below links:
    a. Analyzing Source Code: http://docs.sonarqube.org/display/SONAR/Analyzing+Source+Code
    b. Analyzing with SonarQube Scanner: http://docs.sonarqube.org/display/SCAN/Analyzing+with+SonarQube+Scanner

Customizing the Rule Sets

Some of the rules that are included in the SonarQube language plugins might not be inline with the standards that are being followed in the respective repository. So, in order to avoid the false positive results, the rule sets need to be customized. Below are the steps to customize the rule sets:

1. Login to SonarQube dashboard with admin credentials
2. Click on "Quality Profiles" menu item from the top menu bar
3. Locate the language for which rules are to be customized e.g. for Java, navigate to "Java Profiles" section
4. Click on "Create" link, it will open a popup asking for the profile name
5. Enter the name for new profile and click on "Create" button. This will show the newly created profile in the list of available profiles
6. Now navigate to "Rules" menu from top menu bar. This will show the list of all available rules for the installed language plugins
7. Click on the required language from filter group shown on the left side of page. In the current case, it would be "Java"
8. Click on "Bulk Change" button located at the top right corner of the page
9. Then click on "Activate In" link. It will show a popup asking for the quality profile
10. Type the name of quality profile that we created in step 5 and click on "Apply" button
11. Once the process is complete, click on "Close" link to close the popup
12. Go back to the profile list by clicking on "Quality Profiles" link in top menu
13. Click on "Set as Default" link against the profile, which will set this profile as default for the code analysis that will be done for the specified language
14. Click on the newly created profile, this will show the summary of rules included in the profile
15. Click on the link showing the total number of rules, it will navigate to the list of rules
16. The desired rules can be disabled by clicking on the "Deactivate" button against the rule
17. To change severity for a rule:
    a. Click on the rule, it will open the page showing all the details about the rule
    b. Locate the desired quality profile at the bottom of the page
    c. Click on "Change" button against the quality profile, it will open the popup showing the default severity for the rule
    d. Select the desired severity from the drop down and click on "Save" button
18. For current rule sets configured configured for various languages, please refer:
    a. PHP: SonarQube Scan Report After 2015.05.19 Release With Custom Rule Sets#PHPRules
    b. JavaScript: SonarQube Scan Report After 2015.05.19 Release With Custom Rule Sets#JavaScriptRules
    c. Java: Static Code Analysis for proxy-cs code#cscode-JavaRules

Integrating SonarQube with Jenkins CI

Code analysis using SonarQube can be configured as a job within Jenkins which will be triggered at the time of build creation, or on an ad-hoc basis whenever required. Below steps need to be followed to integrate SonarQube with Jenkins CI:

1. Install SonarQube scanner via Jenkins Update Center. For more details, please refer: https://wiki.jenkins-ci.org/display/JENKINS/Plugins
2. Add SonarQube Server to Jenkins
    a. Log into Jenkins as administrator
    b. Go to "Manage Jenkins" and then click on "Configure System" link
    c. Scroll down to "SonarQube Configuration" section and click on "Add SonarQube" and add the required values in configuration fields
3. Add SonarQube Scanner
    a. Follow steps a and b mentioned in the above section
    b. Scroll down to "SonarQube Runner Configuration" section and click on "Add SonarQube Runner" and add the required values in configuration fields
4. Analyzing with SonarQube Runner
    a. Go to the "Build" section on Jenkins then click on "Add Build Step" and select "Invoke Standalone SonarQube Analysis"
    b. In the configurations, we can point to an existing *sonar-project.properties* file (that we created in step 5 of "Analyzing Source Code with SonarQube Scanner" section)
    c. Alternatively, the project properties can be set at the runtime by configuring them in the "Project Properties" text area in the build configuration
        i. This will be helpful for us in order to set the project properties at the run time
        ii. We don't need to have a properties file checked-in to the repository, and can configure the properties directly on Jenkins
        iii. We can also configure the properties with placeholder for "Project Version" so that we can take the tag as a command input from Huell and use it in configuration

Fore more details on configuring SonarQube with Jenkins and analyzing source code, please refer: http://docs.sonarqube.org/display/SCAN/Analyzing+with+SonarQube+Scanner+for+Jenkins

Impact Analysis

Below is the impact analysis of integrating SonarQube plugin with Jenkins CI Pipeline:

- If SonarQube is run with each and every build then it might consume lot of space on database server to store the scan reports for each build
    - This can be avoided by configuring the SonarQube scan as an optional build parameter, or by triggering SonarQube scan on ad-hoc basis instead of each build (like Checkmarx is configured as of now)
- If SonarQube scan is configured as a parallel child job, then there will be no impact on the build process
    - If the SonarQube scan is scheduled to be integrated with each and every build by default, then it would have to be run as a parallel child job
    - If this is not done as a parallel job and if this is sequential, then it would add to the time consumed by build process